CPSC-406 Report

Automata Theory — Lecture Review & Homework

Nathan Carnnahan

Chapman University

April 9, 2026

Topics Covered

1. Deterministic Finite Automata (DFAs)
2. Formal Languages
3. Language Accepted by a DFA
4. DFA Processing & Design
5. The Extended Transition Function
6. Product Automata
7. NFAs & Determinization
8. Regular Expressions
9. Kleene's Algorithm
10. DFA Minimization
11. The Pumping Lemma
12. Turing Machines & Computability
13. The Halting Problem
14. Decidability Problems
15. Landau Notation (Big O)
16. Complexity Classes (P vs NP)
17. Propositional Logic & SAT

Contents

1 Deterministic Finite Automata **3**
1.1 Representing a DFA . . . 3

2 Formal Languages **3**
2.1 Word Length . . . 4
2.2 Example Languages over $\{0, 1\}$ . . . 4

3 The Language Accepted by a DFA **4**

4 DFA Processing & Design **4**
4.1 Pattern: Words ending with `ab` . . . 5
4.2 Pattern: Parity Tracking . . . 5

5 The Extended Transition Function **5**

6 Product Automata **6**
6.1 Complement . . . 6

7 NFAs & Determinization **6**
7.1 The Power Set Construction . . . 7

8 Regular Expressions **7**
8.1 Operations on Languages . . . 7
8.2 Syntax and Semantics . . . 7
8.3 Equivalence with DFAs and NFAs . . . 8

9 Kleene's Algorithm **8**

10 DFA Minimization **9**
10.1 The Table-Filling Method . . . 9

11 The Pumping Lemma **9**
11.1 Why It Is True . . . 10
11.2 Proof Template . . . 10

12 Turing Machines & Computability **10**
12.1 Why DFAs Are Insufficient . . . 10
12.2 Definition . . . 10
12.3 Example TM: Accepting $\{0^n 1^n\}$ . . . 11
12.4 Instantaneous Descriptions . . . 11
12.5 Computability Hierarchy . . . 11

13 The Halting Problem **11**
13.1 Intuition — Why Self-Reference Breaks Things . . . 12

14 Decidability Problems **12**
14.1 Decidable Problems for DFAs . . . 12
14.2 Undecidable Problems for Turing Machines . . . 13
14.3 The Sharp Contrast . . . 13
14.4 Rice's Theorem (Conceptual) . . . 13

15 Landau Notation (Big O) **13**
15.1 Growth of Functions . . . 14
15.2 Definition . . . 14
15.3 Key Properties . . . 14
15.4 Classifying Algorithms . . . 14

16 Complexity Classes (P vs NP) **15**
16.1 Problems as Languages . . . 15
16.2 The Classes P and NP . . . 15
16.3 Reductions and NP-Completeness . . . 15

17 Propositional Logic & SAT **16**
17.1 Boolean Propositional Logic . . . 16
17.2 The SAT Problem . . . 16
17.3 Conjunctive Normal Form (CNF) . . . 17
17.4 De Morgan's Laws . . . 17
17.5 Resolution . . . 18

Quick Reference Summary **19**

18 Week by Week **21**
18.1 Week 1 . . . 21
18.2 Week 2 . . . 22
18.3 Week 3 . . . 26
18.4 Week 4 . . . 29
18.5 Week 5 . . . 32
18.6 Week 6 . . . 34
18.7 Week 7 . . . 39
18.8 Week 8 . . . 41

19 Synthesis **42**

20 Evidence of Participation **42**

21 Conclusion **42**

1. Deterministic Finite Automata

A **Deterministic Finite Automaton** is a machine that reads a string of characters one symbol at a time and decides whether to *accept* or *reject* it. It is called *deterministic* because at every step there is exactly one possible next state — no guessing.

Definition (DFA). A *deterministic finite automaton* $\mathcal{A} = (Q, \Sigma, \delta, q_0, F)$ consists of:

1. A finite set of *states* Q
2. A finite set of *input symbols* Σ (the alphabet)
3. A *transition function* $\delta \colon Q \times \Sigma \to Q$
4. An *initial state* $q_0 \in Q$
5. A set of *accepting states* $F \subseteq Q$

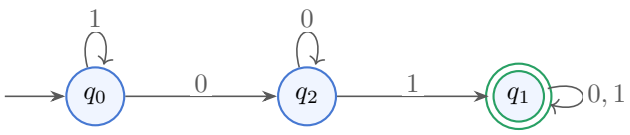
1.1 Representing a DFA

A DFA can be described in two equivalent ways:

- **Transition diagram** — a directed graph where nodes are states, edges are labelled with symbols, the start state has an incoming arrow, and accepting states are drawn with double circles.
- **Transition table** — rows are states, columns are symbols, each cell gives the next state. Start state marked $\to$, accepting states with $*$.

Example 1.1. Consider the DFA with $Q = \{q_0, q_1, q_2\}$, $\Sigma = \{0, 1\}$, start state q_0, $F = \{q_1\}$, and transitions $\delta(q_0, 0) = q_2$, $\delta(q_0, 1) = q_0$, $\delta(q_2, 0) = q_2$, $\delta(q_2, 1) = q_1$, $\delta(q_1, 0) = \delta(q_1, 1) = q_1$.

This DFA accepts any binary string containing the substring `01`.



Transition table:

	0	1
$\to q_0$	q_2	q_0
$* q_1$	q_1	q_1
q_2	q_2	q_1

2. Formal Languages

Key definitions.

- **Alphabet** Σ — any finite set of symbols, e.g. $\{0, 1\}$ or $\{a, b\}$.
- **Word** — any finite sequence of symbols from Σ, including the *empty word* ε.

- Σ^* — the set of all possible words: $\{\varepsilon, 0, 1, 00, 01, 10, 11, \ldots\}$
- **Language** L — any subset $L \subseteq \Sigma^*$.

2.1 Word Length

The *length* of a word is defined inductively:

$$|w| \;:=\; \begin{cases} 0 & \text{if } w = \varepsilon \\ |v| + 1 & \text{if } w = av,\ a \in \Sigma,\ v \in \Sigma^* \end{cases}$$

The *occurrence count* of symbol a in word w is written $|w|_a$.

2.2 Example Languages over $\{0, 1\}$

Language	Description
$L_1 = \{x01y \mid x, y \in \Sigma^*\}$	Words containing `01` as a substring
$L_2 = \{w \mid |w| = 2^n \text{ for some } n\}$	Words whose length is a power of 2
$L_3 = \{w \mid |w|_0 = |w|_1\}$	Words with equal numbers of 0s and 1s

Tip. Languages can be classified by complexity — this is the **Chomsky Hierarchy**. The simplest class, **regular languages**, are exactly those recognisable by DFAs and describable by regular expressions.

3. The Language Accepted by a DFA

A DFA processes a word by starting at q_0 and following transitions one symbol at a time. After reading the last symbol, if the current state is in F the word is *accepted*; otherwise it is *rejected*.

Definition. The *language accepted* by $\mathcal{A}$ is:

$$L(\mathcal{A}) \;:=\; \big\{w \in \Sigma^* \;\big|\; \text{the run of } w \text{ in } \mathcal{A} \text{ ends in some } q \in F\big\}$$

Example 3.1. Tracing $w = 0010$ through the DFA from Section 2:

Step	Symbol	Current State	Next State
Start	—	q_0	—
1	0	q_0	q_2
2	0	q_2	q_2
3	1	q_2	q_1
4	0	q_1	q_1

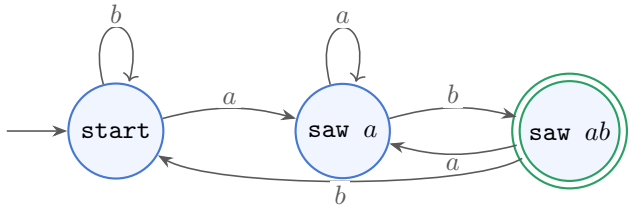
Final state $q_1 \in F$, so $w = 0010$ is **accepted**.

4. DFA Processing & Design

The central design question: *what information does the machine need to remember in order to make the correct accept/reject decision?* Each state encodes a piece of that memory.

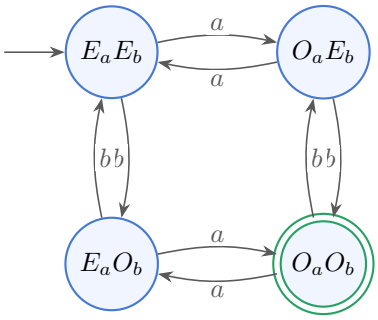
4.1 Pattern: Words ending with `ab`

Track the last one or two characters seen. States: "haven't seen useful prefix", "just saw `a`", "just saw `ab`".



4.2 Pattern: Parity Tracking

To track whether the counts of a's and b's are odd or even, use four states — one per parity combination. The accepting state is (odd a, odd b).



> **Insight.** Parity-tracking DFAs form a grid. Each axis tracks one counter's parity. This generalises: to track any finite combination of properties, make one state per combination.

5. The Extended Transition Function

The standard δ handles one symbol at a time. The **extended transition function** $\widehat{\delta}\colon Q \times \Sigma^* \to Q$ generalises this to entire words.

> **Definition.** Define $\widehat{\delta}$ inductively:
>
> $$\widehat{\delta}(q, w) \;:=\; \begin{cases} q & \text{if } w = \varepsilon \\ \delta\big(\widehat{\delta}(q, x),\, a\big) & \text{if } w = xa,\;\; a \in \Sigma,\;\; x \in \Sigma^* \end{cases}$$

This lets us write the accepted language as $L(\mathcal{A}) = \{w \in \Sigma^* \mid \widehat{\delta}(q_0, w) \in F\}$.

Example 5.1. For a DFA with states $\{1, 2, 3\}$, $\Sigma = \{a, b\}$, start state 1, $F = \{3\}$, computing $\widehat{\delta}(1, baba)$:

Step	Computation	Result
1	$\widehat{\delta}(1, \varepsilon)$	1
2	$\delta(\widehat{\delta}(1, \varepsilon), b) = \delta(1, b)$	1
3	$\delta(\widehat{\delta}(1, b), a) = \delta(1, a)$	2
4	$\delta(\widehat{\delta}(1, ba), b) = \delta(2, b)$	3
5	$\delta(\widehat{\delta}(1, bab), a) = \delta(3, a)$	$3 \in F$ ✓

Word *baba* is **accepted**.

6. Product Automata

Given two DFAs $\mathcal{A}^{(1)}$ and $\mathcal{A}^{(2)}$, we can build a single DFA for their *intersection* or *union* by running both simultaneously.

Definition (Product Automaton). For DFAs $\mathcal{A}^{(1)} = (Q^{(1)}, \Sigma, \delta^{(1)}, q_0^{(1)}, F^{(1)})$ and $\mathcal{A}^{(2)} = (Q^{(2)}, \Sigma, \delta^{(2)}, q_0^{(2)}, F^{(2)})$, define $\mathcal{A} = (Q, \Sigma, \delta, q_0, F)$ by:

$$\begin{aligned} Q &:= Q^{(1)} \times Q^{(2)} \\ \delta\big((p,q),a\big) &:= \big(\delta^{(1)}(p,a),\ \delta^{(2)}(q,a)\big) \\ q_0 &:= (q_0^{(1)}, q_0^{(2)}) \end{aligned}$$

The choice of F determines the operation:

Operation	Accepting states F
Intersection $\cap$	$F^{(1)} \times F^{(2)}$
Union $\cup$	$(F^{(1)} \times Q^{(2)}) \cup (Q^{(1)} \times F^{(2)})$

6.1 Complement

To build a DFA for $\overline{L(\mathcal{A})} = \Sigma^* \setminus L(\mathcal{A})$, simply **swap accepting and non-accepting states** — everything else stays the same.

Tip. When building a product automaton, start from $(q_0^{(1)}, q_0^{(2)})$ and only trace reachable pairs — unreachable states can be ignored.

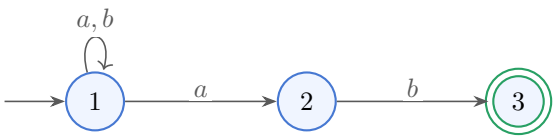
7. NFAs & Determinization

Definition (NFA). A *nondeterministic finite automaton* has the same 5-tuple as a DFA, except the transition function maps to the *power set*:

$$\delta\colon Q \times \Sigma \to \mathcal{P}(Q)$$

A word is accepted if *at least one* path through the NFA ends in an accepting state.

Example 7.1. The NFA below accepts strings that end in `ab`. From state 1, reading `a` nondeterministically leads to $\{1\}$ or $\{1, 2\}$.



7.1 The Power Set Construction

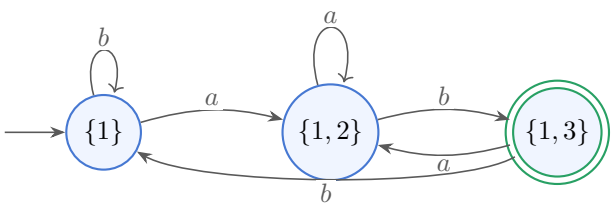
Every NFA can be converted to an equivalent DFA. The states of the new DFA are *subsets* of the NFA's states.

Construction (Power Set Automaton). Given NFA $\mathcal{A} = (Q, \Sigma, \delta, q_0, F)$, define $\mathcal{A}^D = (Q^D, \Sigma, \delta^D, q_0^D, F^D)$ by:

$$\begin{aligned} Q^D &:= \mathcal{P}(Q) \\ \delta^D(S, a) &:= \bigcup_{q \in S} \delta(q, a) \\ q_0^D &:= \{q_0\} \\ F^D &:= \{S \subseteq Q \mid S \cap F \neq \emptyset\} \end{aligned}$$

Example 7.2. NFA: $Q = \{1, 2, 3\}$, $\Sigma = \{a, b\}$, $q_0 = 1$, $F = \{3\}$, with $\delta(1, a) = \{1, 2\}$, $\delta(1, b) = \{1\}$, $\delta(2, b) = \{3\}$, all others $= \emptyset$.

State	a	b	Accepting?
$\to \{1\}$	$\{1, 2\}$	$\{1\}$	No
$\{1, 2\}$	$\{1, 2\}$	$\{1, 3\}$	No
$* \{1, 3\}$	$\{1, 2\}$	$\{1\}$	Yes ($3 \in F$)



Key fact. In principle $|Q^D| = 2^{|Q|}$, but most subsets are unreachable in practice. Always build the table incrementally starting from $\{q_0\}$.

8. Regular Expressions

Regular expressions are a concise algebraic way to *describe* regular languages.

8.1 Operations on Languages

Three fundamental operations:

- **Union:** $L \cup M = \{x \mid x \in L \text{ or } x \in M\}$
- **Concatenation:** $LM = \{vw \mid v \in L,\ w \in M\}$
- **Kleene star:** $L^* = \bigcup_{i=0}^{\infty} L^i$, where $L^0 = \{\varepsilon\}$ and $L^{i+1} = L^i \cdot L$

8.2 Syntax and Semantics

Regular expressions are built inductively.

Base cases:

- $\emptyset$: the empty language; $L(\emptyset) = \emptyset$
- ε: the language $\{\varepsilon\}$
- **a** for any $a \in \Sigma$: the language $\{a\}$

Step cases:

- **Sum** $E + F$: $L(E + F) = L(E) \cup L(F)$
- **Concatenation** EF: $L(EF) = L(E)\,L(F)$
- **Kleene star** E^*: $L(E^*) = L(E)^*$

Operator precedence (high to low): $^* \succ \cdot \succ +$.

So $01^* + 1 = \big((0 \cdot (1^*)) + 1\big)$.

8.3 Equivalence with DFAs and NFAs

From	To	Method
Regex	NFA	Structural induction on the expression
NFA	DFA	Power set construction
DFA	Regex	Kleene's algorithm (Section 10)

9. Kleene's Algorithm

Given a DFA $\mathcal{A}$ with states renamed $Q = \{1, \ldots, n\}$, Kleene's algorithm produces a regular expression $R_{\mathcal{A}}$ with $L(\mathcal{A}) = L(R_{\mathcal{A}})$.

The key objects. For $0 \le k \le n$ and $1 \le i, j \le n$, let $R_{ij}^{(k)}$ be the regular expression describing all paths from state i to state j whose *intermediate* states are all numbered $\le k$.

Base case ($k = 0$): Only direct transitions allowed.

$$L_{ij}^{(0)} = \begin{cases} \{a \in \Sigma \mid \delta(i, a) = j\} & \text{if } i \neq j \\ \{\varepsilon\} \cup \{a \in \Sigma \mid \delta(i, a) = j\} & \text{if } i = j \end{cases}$$

Inductive step ($k - 1 \mapsto k$):

$$R_{ij}^{(k)} \;=\; \underbrace{R_{ij}^{(k-1)}}_{\text{avoid } k} + \underbrace{R_{ik}^{(k-1)}}_{\text{reach } k}\,\underbrace{\big(R_{kk}^{(k-1)}\big)^*}_{\text{loop at } k}\,\underbrace{R_{kj}^{(k-1)}}_{\text{leave } k}$$

Final result:

$$R_{\mathcal{A}} \;=\; \sum_{q \in F} R_{q_0 q}^{(n)}$$

Note. This algorithm is correct but expensive: for n states we compute $O(n^3)$ expressions,

each potentially of exponential length. A more practical alternative is the **state elimination method**.

10. DFA Minimization

Given a DFA, find the equivalent DFA with the **fewest states** by merging states that are indistinguishable.

Definition (State Equivalence). States $p, q \in Q$ are *equivalent*, written $p \sim q$, if:

$$\forall w \in \Sigma^* \colon \widehat{\delta}(p, w) \in F \iff \widehat{\delta}(q, w) \in F$$

That is, no string can distinguish p from q.

10.1 The Table-Filling Method

1. Mark all pairs (p, q) where one is accepting and the other is not.
2. For each unmarked pair (p, q) and each $a \in \Sigma$: if $(\delta(p, a), \delta(q, a))$ is already marked, mark (p, q) too.
3. Repeat step 2 until no new pairs get marked. Unmarked pairs are equivalent.
4. Merge each equivalence class $[q]_\sim$ into a single state. Remove unreachable states.

Construction (Minimised DFA). Define $\mathcal{A}^{\min} = (Q^{\min}, \Sigma, \delta^{\min}, q_0^{\min}, F^{\min})$ by:

$$\begin{aligned} Q^{\min} &:= \{[q]_\sim \mid q \in Q\} \\ \delta^{\min}([q], a) &:= [\delta(q, a)] \\ q_0^{\min} &:= [q_0] \\ F^{\min} &:= \{[q] \mid q \in F\} \end{aligned}$$

Theorem 10.1. Let $\mathcal{A}$ be a DFA with all states reachable. Then $\mathcal{A}^{\min}$ is the (essentially unique) DFA with the same language and the minimal number of states.

11. The Pumping Lemma

Not every language is regular. The Pumping Lemma lets us *prove* a language is **not** regular.

Theorem (Pumping Lemma). Let L be a regular language. Then there exists a constant $n > 0$ such that for every $w \in L$ with $|w| \geq n$, w decomposes as $w = xyz$ satisfying:

1. $y \neq \varepsilon$
2. $|xy| \leq n$
3. $\forall k \geq 0 \colon xy^k z \in L$

11.1 Why It Is True

If $L = L(\mathcal{A})$ for a DFA with n states, any accepted word of length $\geq n$ forces $n+1$ states to be visited. By the **Pigeonhole Principle** some state $p_i = p_j$ is visited twice within the first $n+1$ steps. The substring $y = a_{i+1} \cdots a_j$ forms a loop that can be repeated (or skipped) any number of times.

11.2 Proof Template

1. **Assume** L is regular for contradiction. So L has some pumping number n.
2. **Choose** a specific $w \in L$ with $|w| \geq n$ designed to force a contradiction.
3. **Consider** all decompositions $w = xyz$ satisfying conditions 1 and 2.
4. **Show** that for each split, some $xy^kz \notin L$, contradicting condition 3.
5. **Conclude** L is not regular.

Example 11.1. Show that $L = \{a^n b^n \mid n \geq 0\}$ is not regular.

Proof. Assume L is regular with pumping number n. Choose $w = a^n b^n$ (so $|w| = 2n \geq n$). By condition 2, $xy \subseteq a^n$, so y consists only of a's. By condition 1, $|y| \geq 1$. Pumping with $k = 2$ gives $w' = xy^2z$, which has strictly more a's than b's, so $w' \notin L$. This contradicts condition 3. Therefore L is not regular. □

> **Important.** The Pumping Lemma can only *disprove* regularity. Satisfying the pumping conditions does not prove a language is regular — for that you need to construct a DFA or regular expression.

12. Turing Machines & Computability

DFAs cannot count arbitrarily or perform arithmetic. Turing machines extend the model with an infinite read/write tape.

12.1 Why DFAs Are Insufficient

Consider the language $L = \{10^n 10^m 10^{n+m} \mid n, m \in \mathbb{N}\}$, which encodes addition. This language is not regular (provable via the Pumping Lemma), so DFAs cannot perform even basic arithmetic. The core limitation: a DFA has *finite* memory (finitely many states), so it cannot store arbitrarily large numbers.

What is missing? **Read/write memory.** A Turing machine adds an infinite tape that the machine can both read from and write to, and a head that moves in both directions. This gives it the power to compute anything that is algorithmically computable.

12.2 Definition

> **Definition (Turing Machine).** A TM is a 7-tuple $M = (Q, \Sigma, \Gamma, \delta, q_0, B, F)$ where:
>
> - Q — finite set of states
> - Σ — input alphabet

- Γ — tape alphabet, $\Sigma \subseteq \Gamma$
- $\delta(q, X) = (p, Y, D)$ — in state q reading X: go to p, write Y, move $D \in \{L, R\}$
- q_0 — start state
- $B \in \Gamma \setminus \Sigma$ — blank symbol
- $F \subseteq Q$ — accepting states

Theorem 12.1. Every DFA can be regarded as a TM. If $\delta(p, X) = q$ in the DFA, define $\delta(p, X) = (q, X, R)$ in the TM. When the TM reads the first blank, it halts.

12.3 Example TM: Accepting $\{0^n 1^n\}$

Strategy: match each 0 to a 1 by marking them X and Y respectively.

δ	0	1	X	Y	B
q_0	(q_1, X, R)	—	—	(q_3, Y, R)	—
q_1	$(q_1, 0, R)$	(q_2, Y, L)	—	(q_1, Y, R)	—
q_2	—	—	(q_0, X, R)	(q_2, Y, L)	—
q_3	—	—	—	(q_3, Y, R)	(q_4, B, R)
q_4	(accepting — halt)				

12.4 Instantaneous Descriptions

A configuration of a TM is written as an **ID**: tape contents with the current state to the left of the scanned symbol.

$$q_0\,0011 \;\vdash\; Xq_1\,011 \;\vdash\; X0q_1\,11 \;\vdash\; Xq_2\,0Y1 \;\vdash\; \cdots$$

12.5 Computability Hierarchy

Class	Definition	Example
Regular	Accepted by a DFA	$\{w \mid w \text{ contains } 01\}$
Recursively Enum.	Accepted by some TM	$\{0^n 1^n \mid n \geq 0\}$
Recursive (Decidable)	TM that always halts	Most computable problems
Undecidable	No TM can decide it	The Halting Problem

Key takeaway. Regular $\subset$ Recursive (Decidable) $\subset$ RE $\subset$ All Languages. DFAs sit at the bottom; Turing machines reach RE. But even TMs cannot decide everything — the Halting Problem (next section) is the most famous example.

13. The Halting Problem

The Halting Problem asks: given a program P and an input I, does P eventually terminate on I? We want to know whether a universal "halt-checker" can exist.

The Halting Problem — Specification. Does there exist a program H such that:

1. H takes as input a program P and an input I

2. H always terminates (it never loops forever)
3. $H(P, I) = \texttt{true}$ if P terminates on I
4. $H(P, I) = \texttt{false}$ if P does not terminate on I

Theorem 13.1. The Halting Problem is undecidable. No program H satisfying all four properties above can exist.

Proof. By contradiction.

Step 1. Assume a program H with all four properties exists.

Step 2. Define a new "strange" program S as follows:

$$S(X) \;:=\; \begin{cases} \text{loop forever} & \text{if } H(X, X) = \texttt{true} \\ \text{halt} & \text{if } H(X, X) = \texttt{false} \end{cases}$$

In pseudo-code: `if H(X,X) then while true do skip`.

Note: since H exists and always terminates, the definition of S is well-formed.

Step 3. Ask: what happens when we run $S(S)$? We derive a chain of equivalences:

$$\begin{aligned} H(S, S) = \texttt{true} &\iff S(S) \text{ terminates} && \text{(by definition of } H\text{)} \\ &\iff H(S, S) = \texttt{false} && \text{(by definition of } S\text{)} \end{aligned}$$

This is a contradiction: $H(S, S)$ can be neither `true` nor `false`.

Step 4. Therefore our assumption that H exists is wrong. □

13.1 Intuition — Why Self-Reference Breaks Things

The key trick is **diagonalisation** — we feed the program to itself as input (S applied to S). This creates a self-referential paradox akin to "this sentence is false." Any purported halt-checker H can be weaponised against itself by constructing a program that does the *opposite* of what H predicts.

> **Historical note.** Turing (1937) introduced Turing machines precisely to formalise the notion of "program" and prove this result. The diagonalisation technique mirrors Cantor's proof that the reals are uncountable.

14. Decidability Problems

A natural follow-up: which *specific* questions about automata and Turing machines are decidable or undecidable? The answers differ dramatically depending on the model.

14.1 Decidable Problems for DFAs

Because DFAs have finite state spaces, many questions about them can be answered algorithmically:

Problem	Question	Method
Emptiness	$L(\mathcal{A}) = \emptyset$?	Reachability: is any $q \in F$ reachable from q_0?
Finiteness	$\lvert L(\mathcal{A})\rvert < \infty$?	Check for cycles on paths from q_0 to F
Membership	$w \in L(\mathcal{A})$?	Simulate $\mathcal{A}$ on w — $O(\lvert w\rvert)$ time
Equivalence	$L(\mathcal{A}_1) = L(\mathcal{A}_2)$?	Minimise both and check isomorphism
Universality	$L(\mathcal{A}) = \Sigma^*$?	Complement $\mathcal{A}$ and test emptiness

All of the above are decidable because DFAs have a **finite** number of states — we can exhaustively explore the entire state space.

14.2 Undecidable Problems for Turing Machines

For Turing machines, almost every non-trivial question becomes undecidable. These can be shown undecidable by **reduction** from the Halting Problem.

Problem	Question	Decidable?
Halting	Does TM M halt on input w?	No
Emptiness	$L(M) = \emptyset$?	No
Equivalence	$L(M_1) = L(M_2)$?	No
Regularity	Is $L(M)$ regular?	No
Membership	$w \in L(M)$? (in general)	No

14.3 The Sharp Contrast

Key Insight. The *same question* can be decidable for DFAs but undecidable for TMs. The difference: DFAs have **bounded** computation (finite states, one pass, no writing), while TMs have **unbounded** computation (infinite tape, arbitrary rewinding). The power that makes TMs universal also makes them unpredictable.

14.4 Rice's Theorem (Conceptual)

Any *non-trivial* property of the language recognised by a Turing machine is undecidable. A property is "non-trivial" if some TMs have it and some do not. This means questions like "does this TM accept at least one string?" or "is the language of this TM finite?" are all undecidable.

For the midterm. Remember the sharp contrast — decidable for DFAs, undecidable for TMs. The halting problem proof by contradiction (assume H exists $\to$ build strange program $S \to$ derive $S(S)$ paradox) is the key proof technique.

15. Landau Notation (Big O)

The definition of Big O is less important for its raw mathematical content than for the powerful *metaphor* it creates. This metaphor underpins all of practical software engineering: we classify algorithms into broad clusters according to the resources they consume (time or space) without worrying about exact constants or low-order terms.

15.1 Growth of Functions

Before defining Big O, it is useful to develop intuition for how quickly common functions grow. Listed from slowest to fastest growth:

$$\log(\log n) \ll \log n \ll n \ll n^2 \ll 2^n \ll e^n \ll n! \ll 2^{2^n}$$

Useful identities. $e^{\log n} = n$ and $e^{2\log n} = n^2$, so these seemingly exotic expressions reduce to familiar polynomial growth.

15.2 Definition

Definition (Big O). Given non-negative functions $f, g\colon \mathbb{N} \to \mathbb{R}_{\geq 0}$, we write

$$f \in \mathcal{O}(g)$$

if there exist constants N and M such that $f(n) \leq M \cdot g(n)$ for all $n \geq N$.

Informally, $f \in \mathcal{O}(g)$ means f grows *no faster* than g, up to a constant factor and for large enough inputs.

15.3 Key Properties

For functions $f, g, h\colon \mathbb{N} \to \mathbb{R}_{\geq 0}$:

1. **Reflexivity:** $f \in \mathcal{O}(f)$.
2. **Constant factors vanish:** $\mathcal{O}(c \cdot f) = \mathcal{O}(f)$ for any constant $c > 0$.
3. **Monotonicity:** if $f \leq g$ for large inputs, then $\mathcal{O}(f) \subseteq \mathcal{O}(g)$.
4. **Sum rule:** $\mathcal{O}(f + g) = \mathcal{O}(\max(f, g))$.
5. **Polynomials:** $\mathcal{O}\big(\sum_{i=0}^{k} a_i n^i\big) = \mathcal{O}(n^k)$.
6. **Log vs. polynomial:** $\mathcal{O}(\log n) \subseteq \mathcal{O}(n)$ and $\mathcal{O}(n \log n) \subseteq \mathcal{O}(n^2)$.

Practical consequence. When analysing an algorithm's runtime $T(n)$, we keep only the dominant term and drop constant factors. For example, $T(n) = 5n^3 + 200n^2 + 7$ is $\mathcal{O}(n^3)$.

15.4 Classifying Algorithms

Big O gives us a language for comparing algorithms:

Complexity class	Example algorithms
$\mathcal{O}(1)$	Hash table lookup
$\mathcal{O}(\log n)$	Binary search
$\mathcal{O}(n)$	Linear scan
$\mathcal{O}(n \log n)$	Merge sort, quick sort (average)
$\mathcal{O}(n^2)$	Bubble sort, insertion sort
$\mathcal{O}(2^n)$	Brute-force subset enumeration

16. Complexity Classes (P vs NP)

With Big O notation in hand, we can now classify *problems* (not just algorithms) by their inherent computational difficulty.

16.1 Problems as Languages

Recall that we identify decision problems with formal languages over $\Sigma = \{0, 1\}$: a decision problem P corresponds to a language $L_P \subseteq \Sigma^*$ where $w \in L_P \iff P(w)$ is true.

16.2 The Classes P and NP

Definition.

- P is the class of all languages decidable by a **deterministic** Turing machine in polynomial time, i.e. in $\mathcal{O}(n^k)$ for some k.
- NP is the class of all languages decidable by a **nondeterministic** Turing machine in polynomial time.

Intuitively, problems in P are *efficiently solvable*, while problems in NP have solutions that are *efficiently verifiable* (given a certificate). Since every deterministic TM is a special case of a nondeterministic one, P $\subseteq$ NP.

Whether P = NP is one of the great open problems in mathematics and computer science (a Clay Millennium Prize problem).

Problems in P	Problems in NP
Sorting	Travelling Salesperson (TSP)
Multiplication	Integer Factorisation
Rubik's Cube	Sudoku
DFA membership	Protein Folding

16.3 Reductions and NP-Completeness

Definition (Polynomial-time reduction). We write $L_1 \leq_{\mathrm{P}} L_2$ if there exists a polynomial-time computable function f such that $w \in L_1 \iff f(w) \in L_2$. If such a reduction exists, then L_2 is "at least as hard" as L_1.

Definition (NP-completeness). A language L is **NP-complete** if:

1. $L \in$ NP, and
2. for every $L' \in$ NP, $L' \leq_{\mathrm{P}} L$ (L is **NP-hard**).

Theorem 16.1. If P_1 is NP-complete, $P_2 \in$ NP, and $P_1 \leq_{\mathrm{P}} P_2$, then P_2 is also NP-complete.

Key consequence. If *any* NP-complete problem could be solved in polynomial time, then *every* problem in NP could be solved in polynomial time, i.e. P = NP.

17. Propositional Logic & SAT

A fundamental NP-complete problem is SAT, the **Boolean satisfiability problem**. It asks: given a formula of propositional logic, is there an assignment of truth values that makes it true?

17.1 Boolean Propositional Logic

We work over an alphabet Var of propositional variables $p, q, r, \ldots$ that take values in $\{0, 1\}$ (false/true).

Definition (Boolean formulas). A *formula* is an expression of one of the following forms:

1. $\varphi \vee \psi$ (disjunction / "or")
2. $\varphi \wedge \psi$ (conjunction / "and")
3. $\neg\varphi$ (negation / "not")
4. p (a propositional variable)
5. $b \in \{0, 1\}$ (a truth value)

Definition (Valuation). A *valuation* (or *assignment*) is a function $\mathfrak{J}\colon \mathrm{Var} \to \{0, 1\}$. It extends to all formulas by:

$$\begin{aligned} \mathfrak{J}(\varphi \vee \psi) &= \max(\mathfrak{J}(\varphi), \mathfrak{J}(\psi)) \\ \mathfrak{J}(\varphi \wedge \psi) &= \min(\mathfrak{J}(\varphi), \mathfrak{J}(\psi)) \\ \mathfrak{J}(\neg\varphi) &= 1 - \mathfrak{J}(\varphi) \end{aligned}$$

Derived connectives:

- *Implication:* $\varphi \to \psi := \neg\varphi \vee \psi$
- *Equivalence:* $\varphi \leftrightarrow \psi := (\varphi \to \psi) \wedge (\psi \to \varphi)$

Truth table for the basic connectives:

p	q	$p \vee q$	$p \wedge q$	$\neg p$
0	0	0	0	1
0	1	1	0	1
1	0	1	0	0
1	1	1	1	0

17.2 The SAT Problem

A formula φ is *satisfiable* if there exists a valuation $\mathfrak{J}$ with $\mathfrak{J}(\varphi) = 1$.

Definition (SAT). SAT is the language of (encodings of) all satisfiable Boolean formulas.

Theorem 17.1 (Cook–Levin)**.** SAT is NP-complete.

Consequence. If SAT could be solved in polynomial time, then P = NP. The Cook–Levin theorem was the first proof that an NP-complete problem exists, and it anchors the entire theory of NP-completeness: to show a new problem is NP-complete, one reduces SAT (or another known NP-complete problem) to it.

17.3 Conjunctive Normal Form (CNF)

For algorithmic purposes (e.g. SAT-solving), formulas are often converted to a standard format.

Definition (CNF). A formula is in *conjunctive normal form* if it is a conjunction of *clauses*, where each clause is a disjunction of *literals* (a variable p or its negation $\neg p$):

$$\varphi \;=\; \bigwedge_i \bigvee_j \ell_{i,j}$$

Example 17.1. The formula $\neg p \wedge (p \vee \neg q)$ is in CNF (two clauses: $\neg p$ and $p \vee \neg q$).

The formula $(p \vee \neg q) \wedge (\neg p \vee q \vee r) \wedge p$ is also in CNF.

Every Boolean formula can be converted into an equivalent CNF formula (one that preserves satisfiability). This is important because many SAT-solvers require their input in CNF.

17.4 De Morgan's Laws

De Morgan's laws are essential tools for manipulating Boolean formulas, particularly when converting to CNF or simplifying negations of complex expressions.

De Morgan's Laws. For any formulas φ and ψ:

$$\begin{aligned} \neg(\varphi \vee \psi) &\equiv \neg\varphi \wedge \neg\psi \\ \neg(\varphi \wedge \psi) &\equiv \neg\varphi \vee \neg\psi \end{aligned}$$

In words: negation distributes over disjunction and conjunction by swapping the connective.

These laws generalise to arbitrary numbers of terms:

$$\neg \bigvee_{i=1}^{n} \varphi_i \;\equiv\; \bigwedge_{i=1}^{n} \neg\varphi_i \qquad\qquad \neg \bigwedge_{i=1}^{n} \varphi_i \;\equiv\; \bigvee_{i=1}^{n} \neg\varphi_i$$

Example 17.2. Converting $\neg\big((a \wedge b) \vee (\neg c \wedge d)\big)$ to CNF using De Morgan's laws:

$$\begin{aligned} &\neg\big((a \wedge b) \vee (\neg c \wedge d)\big) \\ &\equiv\; \neg(a \wedge b) \;\wedge\; \neg(\neg c \wedge d) && \text{(De Morgan: } \neg(\cdot \vee \cdot)\text{)} \\ &\equiv\; (\neg a \vee \neg b) \;\wedge\; (c \vee \neg d) && \text{(De Morgan: } \neg(\cdot \wedge \cdot)\text{, double negation)} \end{aligned}$$

The result $(\neg a \vee \neg b) \wedge (c \vee \neg d)$ is already in CNF.

Connection to SAT. De Morgan's laws are the primary mechanism for "pushing negations inward" through a formula. This is a key step in converting any Boolean formula to CNF, which is the standard input format for SAT-solvers. Combined with the distributive law

$\varphi \vee (\psi \wedge \chi) \equiv (\varphi \vee \psi) \wedge (\varphi \vee \chi)$, De Morgan's laws let us systematically rewrite any formula into a conjunction of clauses.

17.5 Resolution

Resolution is a proof technique for determining whether a CNF formula is unsatisfiable. It operates directly on clauses, combining pairs that contain complementary literals.

Definition (Resolvent). Let C_1 and C_2 be clauses such that C_1 contains a literal ℓ and C_2 contains $\neg\ell$. The *resolvent* of C_1 and C_2 on ℓ is:

$$\mathrm{Res}(C_1, C_2) \;=\; (C_1 \setminus \{\ell\}) \;\cup\; (C_2 \setminus \{\neg\ell\})$$

That is, we cancel the complementary pair and merge the remaining literals.

Example 17.3. Let $C_1 = (p \vee q)$ and $C_2 = (\neg p \vee r)$. Resolving on p:

$$\mathrm{Res}(C_1, C_2) \;=\; (q \vee r)$$

Resolution Refutation. Given a CNF formula, repeatedly resolve pairs of clauses and add the resolvents to the clause set. If the **empty clause** $\square$ is derived (all literals cancel out), the formula is **unsatisfiable**.

Example 17.4. Show that $\varphi = (p) \wedge (\neg p \vee q) \wedge (\neg q)$ is unsatisfiable:

1. Resolve (p) and $(\neg p \vee q)$ on $p \;\Rightarrow\; (q)$
2. Resolve (q) and $(\neg q)$ on $q \;\Rightarrow\; \square$

The empty clause is derived, so φ is unsatisfiable.

Why resolution matters. Resolution is *sound* (if $\square$ is derived, the formula is truly unsatisfiable) and *refutation-complete* (if a CNF formula is unsatisfiable, resolution will eventually derive $\square$). It is one of the core techniques underlying modern SAT-solvers.

Quick Reference Summary

Concept	Key Idea
DFA	5-tuple $(Q, \Sigma, \delta, q_0, F)$. Reads input, follows unique transitions, accepts iff final state $\in F$.
Extended $\widehat{\delta}$	$\widehat{\delta}(q, \varepsilon) = q$; $\widehat{\delta}(q, xa) = \delta(\widehat{\delta}(q, x), a)$. Applies transitions to an entire word.
Product automaton	States are pairs. Intersection: $F = F^{(1)} \times F^{(2)}$. Union: accept if either component accepts. Complement: swap F and $Q \setminus F$.
NFA	Like DFA but $\delta : Q \times \Sigma \to \mathcal{P}(Q)$. Accepts if any path ends in F. Equally powerful to DFAs.
Power set construction	NFA→DFA. States are subsets. Build from $\{q_0\}$. Up to 2^n states (often fewer reachable).
Regular expressions	Built from $\emptyset, \varepsilon$, symbols using $+$, $\cdot$, *. Equivalent in power to DFAs and NFAs.
Kleene's algorithm	DFA→regex. Build $R_{ij}^{(k)}$ inductively. Final expression sums over accepting states.
DFA minimisation	Merge equivalent states ($p \sim q$ if no suffix distinguishes them). Table-filling finds all equivalent pairs.
Pumping Lemma	Regular $L \Rightarrow \exists n$ s.t. long words pump. Used only to prove non-regularity via contradiction.
Turing machines	7-tuple. Infinite tape with R/W head. Extends DFAs. Models all algorithmic computation.
Decidability	Decidable = TM that always halts. Undecidable = no such TM exists.
Halting problem	Undecidable. Proof: assume halt-checker H exists, build $S(X)$: loop if $H(X, X)$ is true, halt otherwise. $S(S)$ creates contradiction, so H cannot exist.
DFA decidability	Emptiness, finiteness, membership, equivalence, universality — all decidable via graph search or complement + emptiness.
TM undecidability	Halting, emptiness, equivalence, regularity — all undecidable. Rice's theorem: any non-trivial language property of a TM is undecidable.
Big O notation	$f \in \mathcal{O}(g)$ if $f(n) \leq M \cdot g(n)$ for large n. Classifies algorithms by growth rate, ignoring constants.
P vs NP	P: poly-time solvable. NP: poly-time verifiable. P $\subseteq$ NP; whether P = NP is open.
NP-completeness	L is NP-complete if $L \in$ NP and every NP problem reduces to L. If any NP-complete problem is in P, then P = NP.

Concept	Key Idea
SAT	Boolean satisfiability. NP-complete by Cook–Levin. CNF: conjunction of clauses (disjunctions of literals).
De Morgan's laws	$\neg(\varphi \vee \psi) \equiv \neg\varphi \wedge \neg\psi$ and $\neg(\varphi \wedge \psi) \equiv \neg\varphi \vee \neg\psi$. Used to push negations inward when converting to CNF.
Resolution	Combine clauses with complementary literals ℓ and $\neg\ell$; cancel the pair, merge the rest. Deriving $\square$ proves unsatisfiability.

18. Week by Week

18.1 Week 1

HW1 – DFA Exercises

Exercise 1 We are given two DFAs A_1 and A_2.

Accepted Words Table

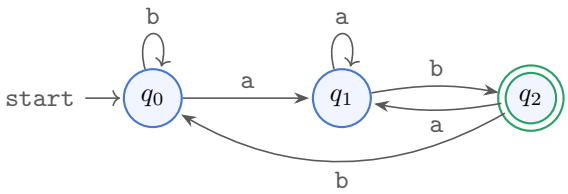
w	Accepted by A_1?	Accepted by A_2?
aaa	No	Yes
aab	Yes	No
aba	No	No
abb	No	No
baa	No	Yes
bab	No	No
bba	No	No
bbb	No	No

Language Descriptions

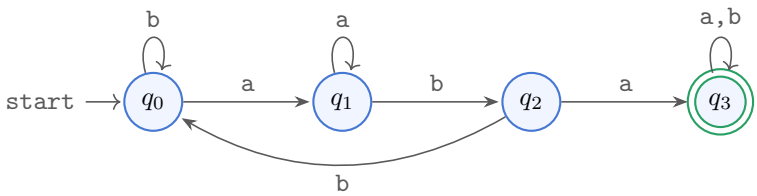
- $L(A_1)$: all strings over $\{a, b\}$ that start with a and end with an odd number of b's.
- $L(A_2)$: all strings over $\{a, b\}$ that end with at least two consecutive a's.

Exercise 2 – Designing DFAs

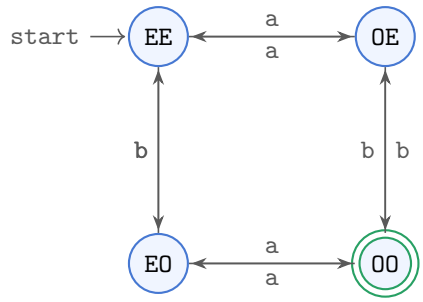
1. Words that end with ab



2. Words that contain aba

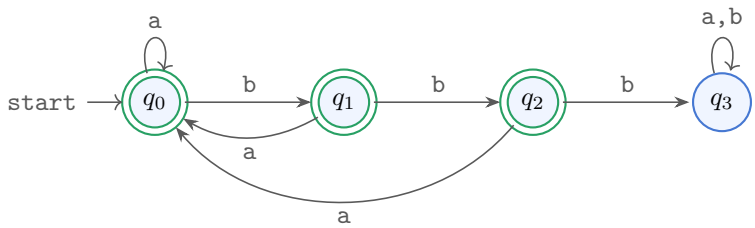


3. Odd number of a's and odd number of b's States represent parity: (a-parity, b-parity).

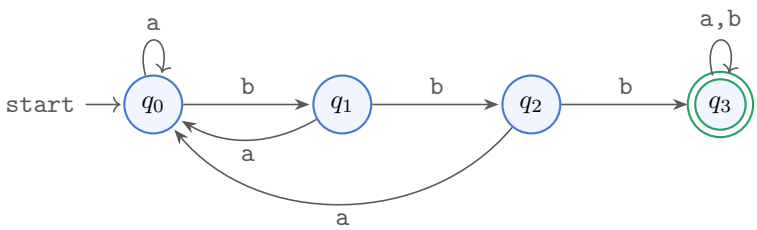


4. Even number of a's and odd number of b's Same automaton as above, but accepting state is EO.

5. Any three consecutive characters contain at least one a Equivalent to forbidding substring bbb.



6. Words that contain bbb



Observation

- Problems 3 and 4 use the same parity structure; only the accepting state changes.
- Problems 5 and 6 use the same "count consecutive b's" structure; one treats reaching three b's as rejection, the other as acceptance.
- Problems 1 and 2 track progress toward matching a pattern.

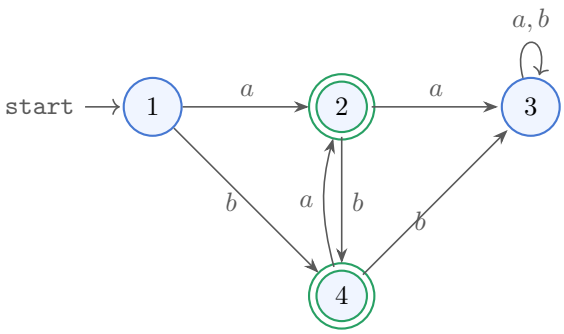
18.2 Week 2

HW2 – Operations on automata

Exercise 1 (Product automata). Throughout, $\Sigma = \{a, b\}$.

1. Description of $L(A^{(1)})$ and $L(A^{(2)})$.

<u>$A^{(1)}$:</u>



The accepting states are $\{2, 4\}$ and state 3 is a non-accepting sink (it loops on both a and b). From state 2, reading a goes to the sink; from state 4, reading b goes to the sink. Thus, once the first symbol is read, the letters must alternate in order to avoid the sink.

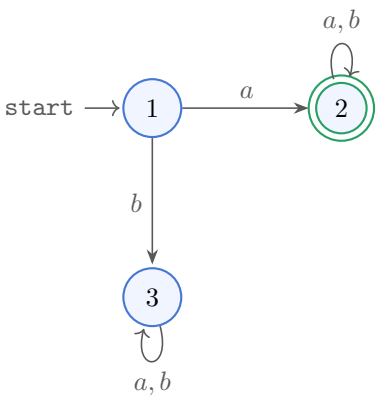
Therefore,

$$L(A^{(1)}) = \{\, w \in \{a, b\}^* : |w| \geq 1 \text{ and } w \text{ contains no substring } aa \text{ or } bb \,\}.$$

Equivalently,

$$L(A^{(1)}) = a(ba)^*(\varepsilon \mid b) \;\cup\; b(ab)^*(\varepsilon \mid a).$$

<u>$A^{(2)}$:</u>



The only accepting state is 2. From state 1, reading b leads to a sink. Reading a moves $1 \to 2$, and from 2 any symbol returns to 1.

Thus a word must:

- start with a,
- have odd length,
- have a in every odd position.

Hence,

$$L(A^{(2)}) = a\big((a \mid b)a\big)^*.$$

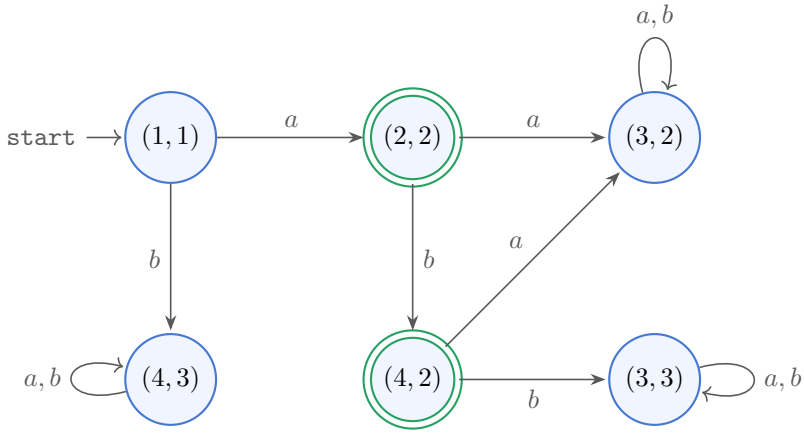
2. Intersection automaton $A = A^{(1)} \times A^{(2)}$.

- States: $Q = Q_1 \times Q_2$
- Start state: $(1, 1)$

- Accepting states:
$$F = F_1 \times F_2 = \{2, 4\} \times \{2\}$$

- Transition function:
$$\delta((p, q), x) = (\delta_1(p, x), \delta_2(q, x))$$

The reachable accepting state is $(2, 2)$.



3. Proof that $L(A) = L(A^{(1)}) \cap L(A^{(2)})$.

For any word w, by induction on prefixes, after reading w the product automaton is in state

$$(\delta_1(1, w), \delta_2(1, w)).$$

Thus A accepts w iff both component automata accept w. Therefore,

$$L(A) = L(A^{(1)}) \cap L(A^{(2)}).$$

4. Construction of union automaton A'.

Keep the same product states and transitions, but define the accepting set as

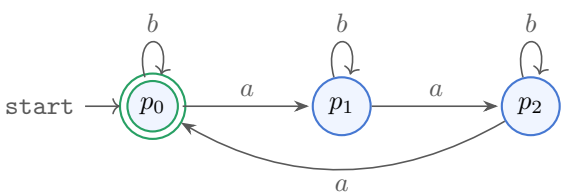
$$F' = (F_1 \times Q_2) \ \cup \ (Q_1 \times F_2).$$

Then A' accepts whenever at least one component accepts, so

$$L(A') = L(A^{(1)}) \cup L(A^{(2)}).$$

Exercise 2 (More automata). 1. Description of $L(B^{(1)})$ and $L(B^{(2)})$.

<u>$B^{(1)}$:</u>



State p_0 is accepting. Reading a cycles

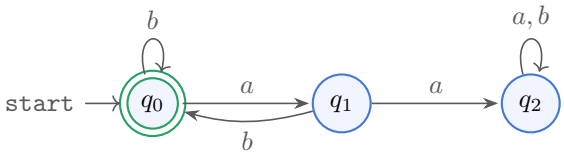
$$p_0 \to p_1 \to p_2 \to p_0,$$

while b loops at each state.

Thus $B^{(1)}$ accepts exactly when the number of a's is divisible by 3:

$$L(B^{(1)}) = \{\, w \in \{a,b\}^* : \#_a(w) \equiv 0 \pmod 3 \,\}.$$

<u>$B^{(2)}$:</u>



State q_0 is accepting. The automaton goes to a sink q_2 if the substring aa occurs. Also, q_1 is non-accepting, so a word ending in a is rejected.

Hence

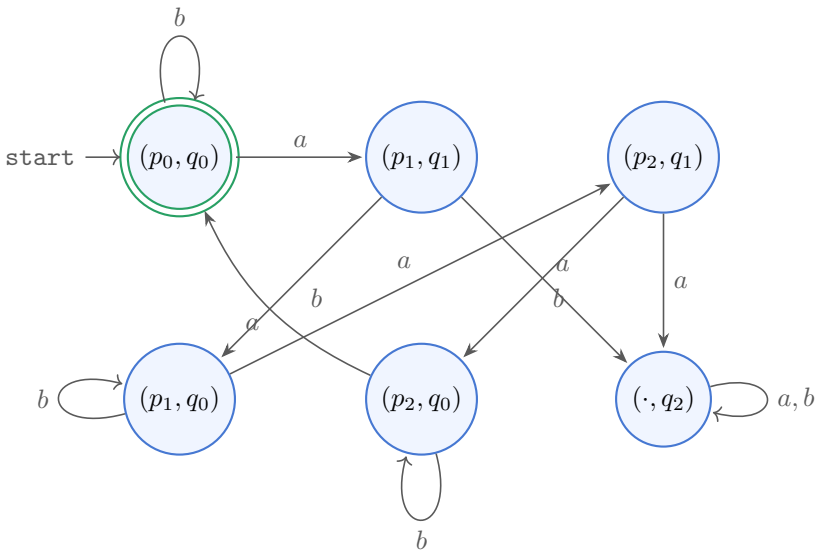
$$L(B^{(2)}) = \{\, w \in \{a,b\}^* : w \text{ has no substring } aa \text{ and does not end with } a \,\}.$$

Equivalently,

$$L(B^{(2)}) = (b \mid ab)^*.$$

2. Intersection automaton $B = B^{(1)} \times B^{(2)}$.

- Start state: (p_0, q_0)
- Accepting states:
$$F = \{(p_0, q_0)\}$$
- Transition function:
$$\delta((p_i, q_j), x) = (\delta_1(p_i, x), \delta_2(q_j, x))$$



3. Proof that $L(B) = L(B^{(1)}) \cap L(B^{(2)})$.

After reading any word w, the product automaton is in state

$$(\delta_1(p_0, w), \delta_2(q_0, w)).$$

Thus B accepts w iff both component automata accept w. Therefore,

$$L(B) = L(B^{(1)}) \cap L(B^{(2)}).$$

4. Construction of union automaton B'.

Using De Morgan's law:

$$L(B^{(1)}) \cup L(B^{(2)}) = \overline{\overline{L(B^{(1)})} \cap \overline{L(B^{(2)})}}.$$

1. Complement $B^{(1)}$ and $B^{(2)}$ by swapping accepting and non-accepting states.
2. Construct their product automaton for intersection.
3. Complement the resulting automaton.

This yields B' such that

$$L(B') = L(B^{(1)}) \cup L(B^{(2)}).$$

Exercise 2.2.7 Let A be a DFA and let q be a state such that $\delta(q, a) = q$ for all input symbols $a \in \Sigma$. We prove by induction on $|w|$ that for all strings $w \in \Sigma^*$, $\hat{\delta}(q, w) = q$.

Proof. **Base case:** $|w| = 0$, so $w = \varepsilon$. By definition of $\hat{\delta}$, $\hat{\delta}(q, \varepsilon) = q$.

Inductive step: Assume that for some $n \geq 0$, for every string $x \in \Sigma^*$ with $|x| = n$ we have $\hat{\delta}(q, x) = q$. Let $w \in \Sigma^*$ be any string with $|w| = n + 1$. Then w can be written as $w = xa$ where $|x| = n$ and $a \in \Sigma$ is the last symbol. Using the recursive definition of $\hat{\delta}$,

$$\hat{\delta}(q, xa) = \delta(\hat{\delta}(q, x), a).$$

By the inductive hypothesis, $\hat{\delta}(q, x) = q$, so

$$\hat{\delta}(q, xa) = \delta(q, a) = q,$$

where the last equality uses the assumption that $\delta(q, a) = q$ for all $a \in \Sigma$. Thus $\hat{\delta}(q, w) = q$ for all w of length $n + 1$.

By induction, $\hat{\delta}(q, w) = q$ for all $w \in \Sigma^*$. □

Interesting Question for Discord (from §2.2.4) Can the same automaton be setup to accept multiple languages? If so, can we use the same automaton hardware to create a multifunctional dfa?

18.3 Week 3

Assignment 3 – NFAs and Determinization

Question 1

For the alphabet $\Sigma = \{a, b, c\}$ consider the NFA A shown in the prompt.

1. Regular expression for $L(A)$. From the start state 0, we may read any number of a's and b's while staying in state 0. At some point we must read an a to move to state 1, and then read exactly one more symbol (b or c) to reach an accepting state. Therefore the language is

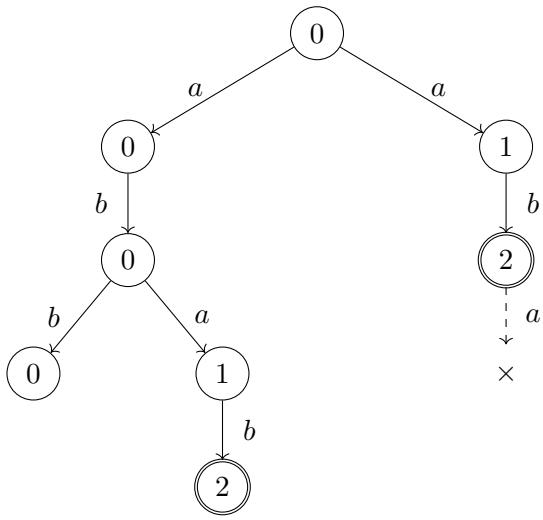
$$L(A) = (a \mid b)^* \, a \, (b \mid c).$$

2. NFA specification $(Q, \Sigma, \delta, q_0, F)$.

- $Q = \{0, 1, 2, 3\}$
- $\Sigma = \{a, b, c\}$
- $q_0 = 0$
- $F = \{2, 3\}$
- Transition function $\delta : Q \times \Sigma \to \mathcal{P}(Q)$ given by:

q	$\delta(q, a)$	$\delta(q, b)$	$\delta(q, c)$
0	$\{0, 1\}$	$\{0\}$	$\emptyset$
1	$\emptyset$	$\{2\}$	$\{3\}$
2	$\emptyset$	$\emptyset$	$\emptyset$
3	$\emptyset$	$\emptyset$	$\emptyset$

3. All paths for $v = abab$. Because $\delta(0, a) = \{0, 1\}$, the first symbol a branches into two threads. The diagram below shows all paths; the accepting path is marked with ✓ and the stuck path is marked with $\times$.



The rightmost branch gets stuck after reading ab because state 2 has no outgoing transitions. The only accepting computation is the leftmost path:

$$0 \xrightarrow{a} 0 \xrightarrow{b} 0 \xrightarrow{a} 1 \xrightarrow{b} 2 \checkmark$$

4. Determinization A^D (powerset construction). The DFA states are subsets of Q. The reachable subsets are

$$\{0\},\ \{0, 1\},\ \{0, 2\},\ \{3\},\ \emptyset.$$

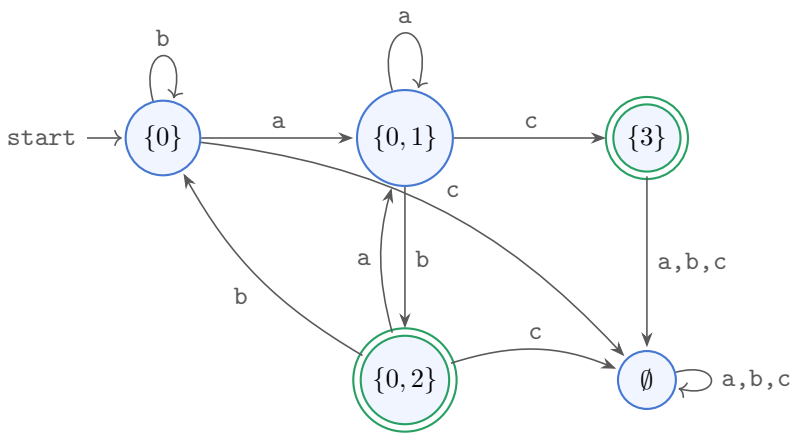
The start state is $\{0\}$ and the accepting DFA states are those subsets containing 2 or 3, i.e.

$$F_D = \{\{0, 2\}, \{3\}\}.$$

Transition table for A^D.

S	$\delta_D(S,a)$	$\delta_D(S,b)$	$\delta_D(S,c)$
$\{0\}$	$\{0,1\}$	$\{0\}$	$\emptyset$
$\{0,1\}$	$\{0,1\}$	$\{0,2\}$	$\{3\}$
$\{0,2\}$	$\{0,1\}$	$\{0\}$	$\emptyset$
$\{3\}$	$\emptyset$	$\emptyset$	$\emptyset$
$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$

Graphical depiction of A^D.



Question 2

For the alphabet $\Sigma = \{0,1\}$ consider the NFA A shown in the prompt.

1. Determinization A^D (powerset construction). The NFA transitions are:

$\delta(q_0,0) = \{q_0\},\ \delta(q_0,1) = \{q_0,q_1\},\ \delta(q_1,1) = \{q_2\},\ \delta(q_2,0) = \{q_1,q_3\},\ \delta(q_3,0) = \delta(q_3,1) = \{q_3\}$

and all other transitions are $\emptyset$.

The reachable DFA states (subsets) are:

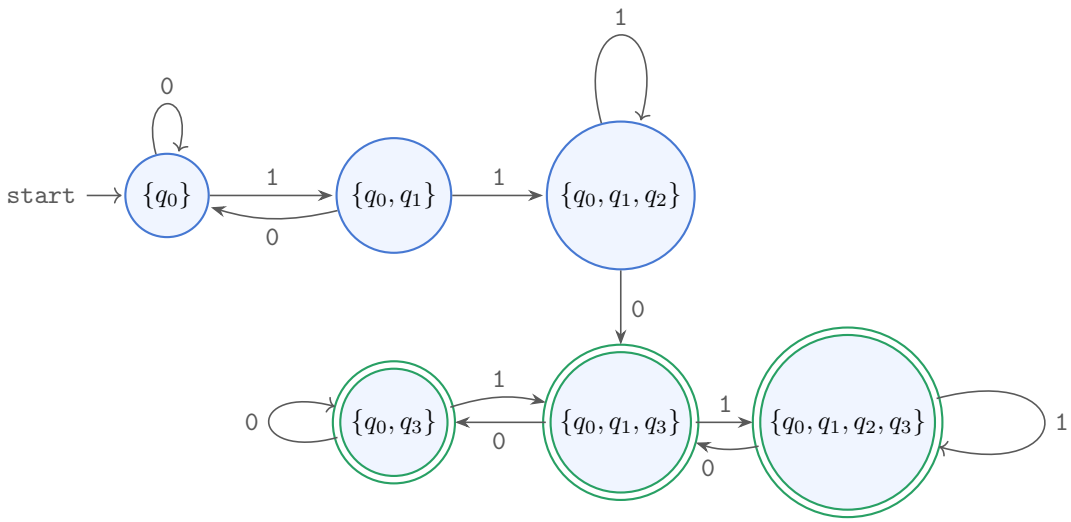
$$\{q_0\},\ \{q_0,q_1\},\ \{q_0,q_1,q_2\},\ \{q_0,q_1,q_3\},\ \{q_0,q_3\},\ \{q_0,q_1,q_2,q_3\}.$$

Start state: $\{q_0\}$. Accepting DFA states are those containing q_3.

Transition table for A^D.

S	$\delta_D(S,0)$	$\delta_D(S,1)$
$\{q_0\}$	$\{q_0\}$	$\{q_0,q_1\}$
$\{q_0,q_1\}$	$\{q_0\}$	$\{q_0,q_1,q_2\}$
$\{q_0,q_1,q_2\}$	$\{q_0,q_1,q_3\}$	$\{q_0,q_1,q_2\}$
$\{q_0,q_1,q_3\}$	$\{q_0,q_3\}$	$\{q_0,q_1,q_2,q_3\}$
$\{q_0,q_3\}$	$\{q_0,q_3\}$	$\{q_0,q_1,q_3\}$
$\{q_0,q_1,q_2,q_3\}$	$\{q_0,q_1,q_3\}$	$\{q_0,q_1,q_2,q_3\}$

Graphical depiction of A^D.

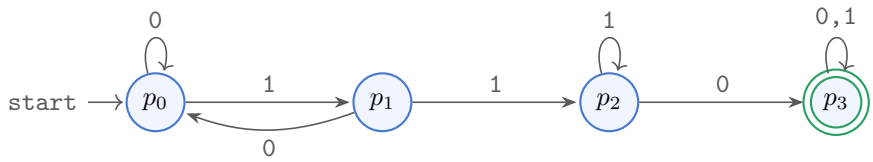


2. Smaller DFA accepting the same language. Intuitively, because q_0 loops on both 0 and 1, the NFA can "start" its nontrivial behavior at any position. To reach the accepting state q_3 we must read the pattern

$$1\,1\,0$$

in consecutive positions (take $q_0 \xrightarrow{1} q_1 \xrightarrow{1} q_2 \xrightarrow{0} q_3$). Once in q_3, the automaton accepts and stays accepting forever. Thus the language is exactly the set of binary strings containing substring 110.

A minimal DFA for "contains 110" has 4 states tracking the longest suffix that is also a prefix of 110:



Here p_3 is the accepting sink (we have seen 110).

18.4 Week 4

HW4 – Kleene's Algorithm and Testing Equivalence

Exercise 1 – Kleene's Algorithm Consider the DFA with states p and q, where p is the start and accepting state, and the transitions are:

$$p \xrightarrow{a} p, \quad p \xrightarrow{b} q, \quad q \xrightarrow{a} p, \quad q \xrightarrow{b} q.$$

Let $1 = p$ and $2 = q$.

For Kleene's algorithm, the base regular expressions are:

$$R_{11}^{(0)} = \varepsilon + a, \qquad R_{12}^{(0)} = b, \qquad R_{21}^{(0)} = a, \qquad R_{22}^{(0)} = \varepsilon + b.$$

Using the recurrence

$$R_{ij}^{(k)} = R_{ij}^{(k-1)} + R_{ik}^{(k-1)} \left(R_{kk}^{(k-1)}\right)^* R_{kj}^{(k-1)},$$

we first compute the $k = 1$ stage (eliminating state 1):

$$\begin{aligned}
R_{11}^{(1)} &= (\varepsilon + a) + (\varepsilon + a)(\varepsilon + a)^*(\varepsilon + a) = a^* \\
R_{12}^{(1)} &= b + (\varepsilon + a)(\varepsilon + a)^* b = a^* b \\
R_{21}^{(1)} &= a + (\varepsilon + b)(\varepsilon + a)^* a = aa^* = a^+ \\
R_{22}^{(1)} &= (\varepsilon + b) + (\varepsilon + b)(\varepsilon + a)^* b = \varepsilon + a^* b
\end{aligned}$$

Then the $k = 2$ stage (eliminating state 2) gives the language expression $R_{11}^{(2)}$, since state 1 $(= p)$ is the only accepting state:

$$\begin{aligned}
R_{11}^{(2)} &= R_{11}^{(1)} + R_{12}^{(1)} \left(R_{22}^{(1)}\right)^* R_{21}^{(1)} \\
&= a^* + a^* b \left(\varepsilon + a^* b\right)^* a^+ \\
&= a^* + a^* b (a^* b)^* a^+
\end{aligned}$$

This simplifies to the regular expression:

$$R = \varepsilon + (a + b)^* a.$$

Thus, the language accepted by the DFA is:

$$L(A) = L(R) = \{\varepsilon\} \cup \{w \in \{a, b\}^* : w \text{ ends in } a\}.$$

Exercise 4.4.2

Repeat Exercise 4.4.1 for the DFA of Fig. 4.15.

From Fig. 4.15, the DFA has states

$$A, B, C, D, E, F, G, H, I$$

with start state A, and accepting states

$$C,\ F,\ I.$$

Its transition table is:

	0	1
A	B	E
B	C	F
C	D	H
D	E	H
E	F	I
F	G	B
G	H	B
H	I	C
I	A	E

(a) Table of distinguishabilities

First, mark every pair consisting of one accepting state and one nonaccepting state. Since the accepting states are C, F, I, all pairs of the form

$$(C, x),\ (F, x),\ (I, x) \quad \text{where } x \notin \{C, F, I\},$$

are immediately distinguishable.

Applying the table-filling algorithm to the remaining pairs shows that the only indistinguishable pairs are:

$$\begin{gathered}(A, D),\ (A, G),\ (D, G),\\ (B, E),\ (B, H),\ (E, H),\\ (C, F),\ (C, I),\ (F, I).\end{gathered}$$

Therefore the equivalence classes are

$$[A] = \{A, D, G\}, \qquad [B] = \{B, E, H\}, \qquad [C] = \{C, F, I\}.$$

(b) Minimum-state equivalent DFA

Let

$$S_1 = \{A, D, G\}, \qquad S_2 = \{B, E, H\}, \qquad S_3 = \{C, F, I\}.$$

Since A is the start state, the start state of the minimized DFA is S_1. Since C, F, I are accepting states, S_3 is the only accepting state.

Now compute transitions using any representative from each class:

$$\delta(S_1, 0) = S_2 \quad \text{since} \quad A \xrightarrow{0} B,\ D \xrightarrow{0} E,\ G \xrightarrow{0} H,$$

and all of B, E, H lie in S_2.

$$\delta(S_1, 1) = S_2 \quad \text{since} \quad A \xrightarrow{1} E,\ D \xrightarrow{1} H,\ G \xrightarrow{1} B,$$

and all of E, H, B lie in S_2.

$$\delta(S_2, 0) = S_3 \quad \text{since} \quad B \xrightarrow{0} C,\ E \xrightarrow{0} F,\ H \xrightarrow{0} I,$$

and all of C, F, I lie in S_3.

$$\delta(S_2, 1) = S_3 \quad \text{since} \quad B \xrightarrow{1} F,\ E \xrightarrow{1} I,\ H \xrightarrow{1} C.$$

$$\delta(S_3, 0) = S_1 \quad \text{since} \quad C \xrightarrow{0} D,\ F \xrightarrow{0} G,\ I \xrightarrow{0} A,$$

and all of D, G, A lie in S_1.

$$\delta(S_3, 1) = S_2 \quad \text{since} \quad C \xrightarrow{1} H,\ F \xrightarrow{1} B,\ I \xrightarrow{1} E,$$

and all of H, B, E lie in S_2.

So the minimized DFA is:

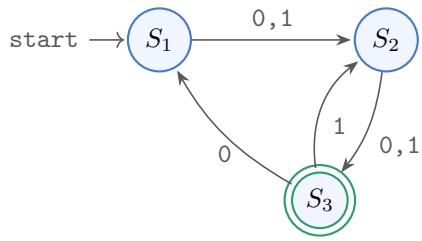
	0	1
S_1	S_2	S_2
S_2	S_3	S_3
S_3	S_1	S_2

where

$$S_1 = \{A, D, G\}, \qquad S_2 = \{B, E, H\}, \qquad S_3 = \{C, F, I\}.$$

Thus, the minimum-state DFA equivalent to the DFA in Fig. 4.15 has $\boxed{3}$ states.

Minimized DFA diagram



Question When applying Kleene's algorithm for the exercise, do we need to explicitly compute every $R_{ij}^{(k)}$ value for all i, j, k, or is it sufficient to compute only the expressions necessary to derive the final regular expression describing the language?

18.5 Week 5

Pumping Lemma Exercises

We use the pumping lemma to show that each of the following languages is not regular. Recall that the pumping lemma states that if a language L is regular, then there exists a pumping length p such that any string $s \in L$ with $|s| \geq p$ can be written as $s = xyz$ where:

- $|xy| \leq p$
- $|y| > 0$
- $xy^iz \in L$ for all $i \geq 0$

We will assume each language is regular and derive a contradiction.

$L_1 = \{a^n b^m \mid n, m \in \mathbb{N}, n > m \geq 0\}$ Assume L_1 is regular and let p be the pumping length. Choose the string:

$$s = a^{p+1} b^p$$

Clearly $s \in L_1$ because $n = p + 1$ and $m = p$ so $n > m$.

By the pumping lemma, $s = xyz$ where $|xy| \leq p$ and $|y| > 0$. Since the first p characters are all a's, the substring y must consist only of a's.

Let $y = a^k$ where $k > 0$.

Now pump down with $i = 0$:

$$xz = a^{p+1-k}b^p$$

Now the number of a's is $p + 1 - k$ which is $\leq p$. This means the number of a's is no longer greater than the number of b's. Therefore $xz \notin L_1$.

This contradicts the pumping lemma, so L_1 is not regular.

$L_2 = \{a^{n^2} \mid n \geq 1\}$ Assume L_2 is regular and let p be the pumping length.

Choose the string:

$$s = a^{p^2}$$

Since p^2 is a perfect square, $s \in L_2$.

Because $|xy| \leq p$, the substring y consists only of a's. Let $y = a^k$ with $k > 0$.

Pump the string with $i = 2$:

$$xy^2z = a^{p^2+k}$$

However, $p^2 + k$ is not guaranteed to be a perfect square since $1 \leq k \leq p$. Therefore the pumped string is not of the form a^{n^2}.

Thus $xy^2z \notin L_2$, contradicting the pumping lemma. Therefore L_2 is not regular.

$L_3 = \{a^p \mid p \textbf{ is prime}\}$ Assume L_3 is regular and let p be the pumping length.

Choose a prime number $q > p$ and let:

$$s = a^q$$

Clearly $s \in L_3$.

Again $|xy| \leq p$ so $y = a^k$ for some $k > 0$.

Pump the string with $i = q + 1$:

$$xy^{q+1}z = a^{q+qk}$$

The exponent becomes:

$$q + qk = q(1 + k)$$

This number is composite since it has factors q and $(1 + k)$ with $k \geq 1$. Therefore the pumped string is not prime length.

Thus $xy^{q+1}z \notin L_3$, contradicting the pumping lemma. Therefore L_3 is not regular.

$L_4 = \{a^k b^m a^{k+m} \mid k, m \geq 0\}$ Assume L_4 is regular and let p be the pumping length.

Choose:

$$s = a^p b^p a^{2p}$$

This string belongs to L_4 since $k = p$ and $m = p$, so the final block has length $k + m = 2p$.

Because $|xy| \leq p$, the substring y lies entirely within the first block of a's.

Let $y = a^j$ where $j > 0$.

Pump down with $i = 0$:

$$xz = a^{p-j} b^p a^{2p}$$

Now the first block has $p - j$ a's while the final block still has $2p$ a's. However the language requires the final block to have exactly $k + m$ a's, which should now be:

$$(p - j) + p = 2p - j$$

But the final block still has $2p$ a's, so the string no longer satisfies the language definition.

Thus $xz \notin L_4$, contradicting the pumping lemma. Therefore L_4 is not regular.

Discord Question One question I had while studying the pumping lemma is how we choose the correct string s to prove a language is not regular. In many examples the string is chosen very strategically (such as choosing a perfect square, a prime length, or balancing parts of the string).

Is there a general strategy for selecting the best string s, or is it mostly based on recognizing patterns in the language definition?

18.6 Week 6

HW6 – Turing Machines and Decidability

Exercise A – Basic Turing Machines

1. TM for $L = \{10^n\}$ that returns 10^{n+1} Idea: move to the end of the string and append a 0.

States: q_0 (start), q_1, q_{acc}

δ	1	0	B
q_0	$(q_1, 1, R)$	–	–
q_1	–	$(q_1, 0, R)$	$(q_{acc}, 0, R)$

Explanation:

- Verify first symbol is 1
- Move right across all 0's
- On blank, write 0 and halt

Example:

$$100 \vdash_M^* 1000$$

2. TM for $L = \{10^n\}$ that returns 1 Idea: erase all 0's after the leading 1.

States: q_0, q_1, q_{acc}

δ	1	0	B
q_0	$(q_1, 1, R)$	–	–
q_1	–	(q_1, B, R)	(q_{acc}, B, R)

Explanation:

- Keep the initial 1
- Replace each 0 with blank
- Halt at end

Example:

$$1000 \vdash_M^* 1$$

3. TM that swaps all 0's and 1's States: q_0, q_{acc}

δ	0	1	B
q_0	$(q_0, 1, R)$	$(q_0, 0, R)$	(q_{acc}, B, R)

Example:

$$10110 \vdash_M^* 01001$$

Exercise B – More Constructions

1. Unary addition: $10^n10^m \to 10^n10^m10^{n+m}$ The assignment requires the output to *append* a third block 10^{n+m} after the original input, leaving 10^n10^m intact.

Idea:

- Use two passes. In the first pass, scan the input to count the total number of 0's (there are $n + m$ of them).
- In practice, a clean single-pass approach is: find the end of the input, append a 1, then copy all $n + m$ zeros from the input to the end.
- Concretely: seek to the first blank after the last 0; write 1; then make repeated passes, each time marking one 0 in the original string as X and appending one 0 at the end, until all 0's are marked; finally restore the X's to 0's.

States: q_0 (scan to end), q_1 (write separator 1), q_2 (seek leftmost unmarked 0), q_3 (seek end to append 0), q_4 (return left), q_5 (restore $X \to 0$), q_{acc}

δ	1	0	X	B	(appended) 0
q_0	$(q_0, 1, R)$	$(q_0, 0, R)$	–	(q_1, B, L)	–
q_1	–	$(q_1, 0, L)$	–	–	–

Because the full construction requires many states and multiple passes, we describe the algorithm at the state level and give the key transitions above; the remaining transitions follow the standard "mark-and-copy" pattern used for similar TMs in the notes.

Example:

$$1\underbrace{00}_{n=2}1\underbrace{000}_{m=3} \vdash_M^* 1\underbrace{00}_{n=2}1\underbrace{000}_{m=3}1\underbrace{00000}_{n+m=5}$$

2. Doubling a string: $w \to ww$ Idea:

- Mark symbols ($0 \to X$, $1 \to Y$)
- Copy them to the end
- Restore symbols

Key transitions:

$$0 \to (X, R)$$
$$1 \to (Y, R)$$

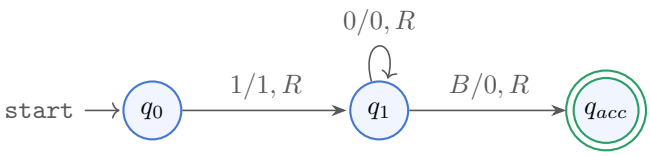
Then:

- Move to end
- Write corresponding symbol
- Return and restore

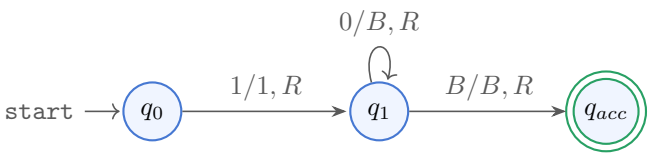
(This construction uses multiple passes and states.)

Exercise C – Transition Diagrams In transition diagrams for TMs, each arc is labelled a/b, D where a is the symbol read, b is the symbol written, and $D \in \{L, R\}$ is the head direction.

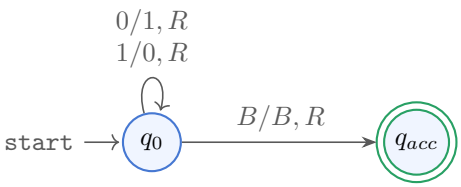
A.1 — Accept 10^n, return 10^{n+1}



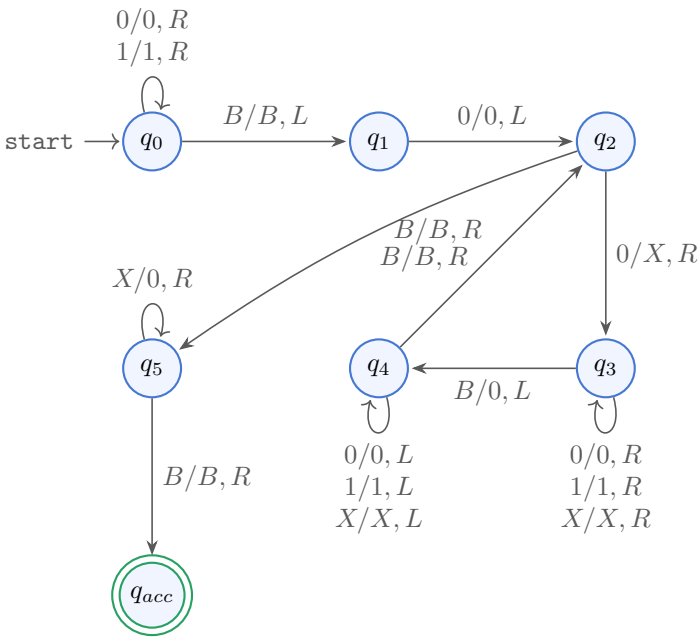
A.2 — Accept 10^n, return 1



A.3 — Swap all 0's and 1's



B.1 — Unary addition (append 10^{n+m}) The diagram below shows the high-level states. States q_2–q_4 implement the mark-and-copy loop; q_5 restores marked symbols.



Exercise D – TM that accepts but never halts Goal: accept strings with at least one 1, but never halt.

States: q_0, q_1. **Accepting state:** $F = \{q_1\}$. Start state: q_0.

δ	0	1	B
q_0	$(q_0, 0, R)$	$(q_1, 1, R)$	(q_0, B, R)
q_1	$(q_1, 0, R)$	$(q_1, 1, R)$	(q_1, B, R)

Explanation:

- If a 1 is found, move to q_1
- Both states loop forever (no halting)

Thus:

- Strings containing 1 are accepted (enter q_1)
- Machine never halts

Exercise E – Multitape Turing Machines

1. Convert $w \to 0^n 1^m$ 2-tape TM:

- Tape 1: input
- Tape 2: output

Algorithm:

- Scan input
- Write 0 for each 0
- Then write 1 for each 1

Example:

$$101001 \to 000111$$

2. Unary multiplication: $10^n 10^m \to 10^{nm}$ Idea:

- For each 0 in first block:
  - Copy second block

This simulates repeated addition.

Exercise 1 – Decidability

1. **True.** Given deciders M_1 for L_1 and M_2 for L_2, construct a TM M that on input w: runs M_1 on w; if M_1 accepts, accept; otherwise run M_2 on w and return its answer. Both M_1 and M_2 always halt, so M always halts. M accepts w iff $w \in L_1$ or $w \in L_2$, i.e. iff $w \in L_1 \cup L_2$.
2. **True.** Given a decider M for L, construct $\overline{M}$ that runs M and swaps accept/reject. Since M always halts, so does $\overline{M}$, and it accepts exactly the strings not in L, i.e. $\overline{L}$.
3. **True.** $\varepsilon \in L^*$ always, so accept immediately on empty input. For a non-empty input w, try all $2^{|w|-1}$ ways to partition $w = w_1 w_2 \cdots w_k$ and for each partition run the decider for L on every piece. Accept if some partition has every piece in L; reject otherwise. There are finitely many partitions and the decider for L always halts, so this procedure always terminates.
4. **True.** Given recognizers M_1 for L_1 and M_2 for L_2, construct a TM M that on input w *dovetails* (interleaves) the simulations: run M_1 for one step, then M_2 for one step, then M_1 for two steps, and so on. If either machine ever accepts, M accepts. Since any $w \in L_1 \cup L_2$ is accepted by at least one of M_1, M_2 in finitely many steps, M will eventually accept it. Hence $L_1 \cup L_2$ is r.e.
5. **False.** The halting language $HALT = \{(M, w) \mid M \text{ halts on } w\}$ is r.e. (simulate M on w; accept if it halts) but its complement $\overline{HALT}$ is not r.e. This is a standard result: if both $HALT$ and $\overline{HALT}$ were r.e., then $HALT$ would be decidable, contradicting the undecidability of the halting problem.
6. **True.** Given a recognizer M for L, we build a recognizer for L^*. On input w, enumerate all partitions of w into non-empty pieces and dovetail the simulation of M on each piece across all partitions simultaneously. Accept as soon as all pieces in some partition are accepted by M. Since any $w \in L^*$ has a valid decomposition into words of L, each accepted in finite time, the machine will eventually find and confirm that partition.

Exercise 2 – Classification

1. $L_1 = \{M \mid M \text{ halts on itself}\}$
   - r.e.: yes
   - decidable: no
   - co-r.e.: no
2. $L_2 = \{(M, w) \mid M \text{ halts on } w\}$
   - r.e.: yes
   - decidable: no
   - co-r.e.: no

3. $L_3 = \{(M, w, k) \mid M \text{ halts in } \leq k \text{ steps}\}$
   - decidable: yes
   - r.e.: yes
   - co-r.e.: yes

Discord Question (Week 6) **Question:** Does it ever make practical or theoretical sense to design a TM that *accepts without halting*, and if so, why would we prefer this over a TM that halts upon acceptance? Is there a class of languages or problems where non-halting acceptance is actually *necessary*, or can we always convert such a machine into one that halts as soon as it accepts?

18.7 Week 7

HW7 – Decidability and Landau Notation

Exercise 2 – Problems on Decidability We classify each language as decidable, recursively enumerable (r.e.), or co-recursively enumerable (co-r.e.).

1. $L_1 := \{\, M \mid M$ **halts on itself**$\}$ This language is **recursively enumerable**, but **not decidable**.

Why it is r.e.: Given a Turing machine M, we can simulate M on its own encoding $\langle M \rangle$. If M eventually halts on $\langle M \rangle$, then our simulator will also halt and accept. If M never halts on $\langle M \rangle$, then the simulator runs forever. Therefore L_1 is recursively enumerable.

Why it is not decidable: This is a version of the halting problem. If we could decide whether M halts on itself, then we would be able to solve a halting-type problem algorithmically, which is impossible. Thus L_1 is undecidable.

About the complement: Since L_1 is r.e. but not decidable, its complement cannot also be r.e. Otherwise both L_1 and $\overline{L_1}$ would be r.e., which would imply that L_1 is decidable. So $\overline{L_1}$ is **not** r.e., meaning L_1 is **not co-r.e.**

$$\boxed{L_1 \text{ is r.e. but neither decidable nor co-r.e.}}$$

2. $L_2 := \{\, (M, w) \mid M$ **halts on the word** $w \,\}$ This language is also **recursively enumerable**, but **not decidable**.

Why it is r.e.: Given (M, w), simulate M on input w. If M halts, accept. If M does not halt, the simulation continues forever. Hence L_2 is recursively enumerable.

Why it is not decidable: This is exactly the standard halting problem, which is known to be undecidable.

About the complement: For the same reason as above, $\overline{L_2}$ is not recursively enumerable. If both L_2 and $\overline{L_2}$ were r.e., then L_2 would be decidable, contradicting undecidability.

$$\boxed{L_2 \text{ is r.e. but neither decidable nor co-r.e.}}$$

3. $L_3 := \{\, (M, w, k) \mid M$ **halts on** w **in at most** k **steps** $\}$ This language is **decidable**.

Decision procedure: Given (M, w, k), simulate M on input w for exactly k steps.

- If M halts within those k steps, accept.
- If it has not halted after k steps, reject.

This always terminates, because k is a fixed finite bound. Therefore L_3 is decidable.

Since every decidable language is also both r.e. and co-r.e., we conclude:

$$\boxed{L_3 \text{ is decidable (and therefore also r.e. and co-r.e.)}}$$

Exercise 4 – Landau Notation We compare the following asymptotic classes.

1. $\mathcal{O}(n)$ **and** $\mathcal{O}(\sqrt{n})$ Since $\sqrt{n} \leq n$ for all $n \geq 1$, every function in $\mathcal{O}(\sqrt{n})$ is also in $\mathcal{O}(n)$. However, not every linear function is bounded by a constant multiple of $\sqrt{n}$.

$$\boxed{\mathcal{O}(\sqrt{n}) \subsetneq \mathcal{O}(n)}$$

2. $\mathcal{O}(n^2)$ **and** $\mathcal{O}(2^n)$ Exponential growth dominates polynomial growth. In particular, $n^2 = O(2^n)$, but $2^n \notin O(n^2)$.

$$\boxed{\mathcal{O}(n^2) \subsetneq \mathcal{O}(2^n)}$$

3. $\mathcal{O}(\log n)$ **and** $\mathcal{O}(\log n \cdot \log n)$ We have $\log n \leq (\log n)^2$ for all sufficiently large n, so every function in $\mathcal{O}(\log n)$ is also in $\mathcal{O}((\log n)^2)$. The reverse inclusion does not hold in general.

$$\boxed{\mathcal{O}(\log n) \subsetneq \mathcal{O}((\log n)^2)}$$

4. $\mathcal{O}(2^n)$ **and** $\mathcal{O}(3^n)$ Because $2^n \leq 3^n$ for all n, every function in $\mathcal{O}(2^n)$ is also in $\mathcal{O}(3^n)$. But 3^n grows strictly faster than 2^n, so the inclusion is proper.

$$\boxed{\mathcal{O}(2^n) \subsetneq \mathcal{O}(3^n)}$$

5. $\mathcal{O}(\log_2 n)$ **and** $\mathcal{O}(\log_3 n)$ By the change-of-base formula,

$$\log_2 n = \frac{\log_3 n}{\log_3 2}.$$

Thus $\log_2 n$ and $\log_3 n$ differ only by a constant factor, so they define the same asymptotic class.

$$\boxed{\mathcal{O}(\log_2 n) = \mathcal{O}(\log_3 n)}$$

Discord Question One thing I found interesting in this homework is the relationship between decidable, recursively enumerable, and co-recursively enumerable languages.

My question is: if a language is known to be recursively enumerable, what are the most common proof strategies for showing that its complement is *not* recursively enumerable? Is the usual approach to reduce from the complement of the halting problem, or are there other standard techniques that are used often in practice?

18.8 Week 8

HW8 – Propositional Logic and SAT

Exercise 1 – Rewriting formulas in CNF

1. $\varphi_1 := \neg\big((a \wedge b) \vee (\neg c \wedge d)\big)$ Apply De Morgan's law to distribute the outer negation:

$$\begin{aligned}\varphi_1 &= \neg(a \wedge b) \ \wedge\ \neg(\neg c \wedge d)\\ &= (\neg a \vee \neg b) \ \wedge\ (c \vee \neg d)\end{aligned}$$

2. $\varphi_2 := \neg\big((p \vee q) \to (r \wedge \neg s)\big)$ First replace the implication using $\alpha \to \beta \equiv \neg\alpha \vee \beta$:

$$\varphi_2 = \neg\big(\neg(p \vee q) \vee (r \wedge \neg s)\big)$$

Apply De Morgan's law to the outer negation:

$$= (p \vee q) \ \wedge\ \neg(r \wedge \neg s)$$

Apply De Morgan again to the second term:

$$= (p \vee q) \ \wedge\ (\neg r \vee s)$$

Exercise 2 – Satisfiability

1. $\psi_1 := (a \vee \neg b) \wedge (\neg a \vee b) \wedge (\neg a \vee \neg b)$ **Satisfiable.** Set $a = 0,\ b = 0$:

- $(0 \vee 1) = 1$
- $(1 \vee 0) = 1$
- $(1 \vee 1) = 1$

All clauses are satisfied.

2. $\psi_2 := (\neg p \vee q) \wedge (\neg q \vee r) \wedge \neg(\neg p \vee r)$ **Not satisfiable.**

First simplify the third clause using De Morgan:

$$\neg(\neg p \vee r) \ \equiv\ p \wedge \neg r$$

This forces $p = 1$ and $r = 0$. Substituting into the remaining clauses:

- $(\neg p \vee q) = (0 \vee q) = q$, so $q = 1$ is forced.
- $(\neg q \vee r) = (0 \vee 0) = 0$. Contradiction.

No satisfying assignment exists.

3. $\psi_3 := (x \vee y) \wedge (\neg x \vee y) \wedge (x \vee \neg y) \wedge (\neg x \vee \neg y)$ **Not satisfiable.**

We can show this by resolution:

- Resolve $(x \vee y)$ and $(\neg x \vee y)$ on x: resolvent is (y).
- Resolve $(x \vee \neg y)$ and $(\neg x \vee \neg y)$ on x: resolvent is $(\neg y)$.
- Resolve (y) and $(\neg y)$ on y: resolvent is $\square$ (the empty clause).

Deriving $\square$ proves the formula is unsatisfiable.

Discord Question Something I noticed in Exercise 2 is how resolution can serve as a systematic alternative to truth-table enumeration for proving unsatisfiability. Is resolution always more efficient than brute-force checking of all 2^n assignments, or are there families of formulas where resolution proofs are exponentially long?

19. Synthesis

20. Evidence of Participation

21. Conclusion

References

[1] John E. Hopcroft, Rajeev Motwani, and Jeffrey D. Ullman, *Introduction to Automata Theory, Languages, and Computation*, 3rd ed., Pearson, 2006.